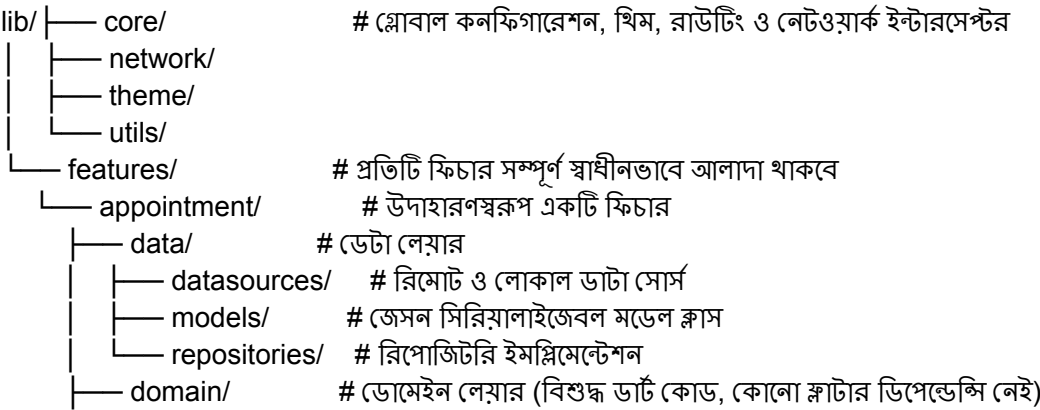


১. প্রজেক্টের বর্তমান অবস্থা এবং বটলেনেক্স চার্ট (Current State & Bottlenecks Chart)

লেয়ার / কম্পোনেন্ট	বর্তমান অবস্থা	সমস্যা ও ব্লকসমূহ
ফোল্ডার স্ট্রাকচার	ক্লাট ও মিক্সড (যেমন: controllers/, views/, repositories/ আলাদা রাখা হয়েছে)	কোনো কঠোর লেয়ার বা ডোমেইন বাউন্ডারি নেই। ফিচার বড় হলে মেনেটেন করা অসম্ভব হয়ে পড়বে।
আর্কিটেকচার	স্ট্যান্ডার্ড এমভিএম (MVVM) বা প্রপার ক্লিন আর্কিটেকচার অনুপস্থিত	বিজনেস লজিক এবং ইউআই লজিক একে অপরের সাথে মিশে যাওয়ার ঝুঁকি রয়েছে, যা ইউনিট টেস্ট লেখা কঠিন করে তোলে।
স্টেট ম্যানেজমেন্ট	কাস্টম কন্ট্রোলার (controllers/ ফোল্ডারে) ব্যবহার করা হচ্ছে	স্টেট ম্যানেজমেন্টের নির্দিষ্ট কোনো গাইডলাইন বা স্ট্যান্ডার্ড না থাকায় মেমোরি লিক এবং অপ্রত্যাশিত রিবিভিউ হতে পারে।
ডিপেন্ডেন্সি ম্যানেজমেন্ট	অবজেক্ট ক্রিয়েশনের সুনির্দিষ্ট নিয়ম বা কন্টইনারের অভাব থাকতে পারে	কোড কাপলিং বেশি থাকে, ফলে স্কেল করা বা মক টেস্ট করা দুরূহ হয়।

২. কাঙ্ক্ষিত লক্ষ্য: প্রডাকশন-গ্রেড আর্কিটেকচার (Target Architecture Chart)

প্রজেক্টটিকে যে স্তরে উন্নীত করতে হবে তা একটি আধুনিক Clean Architecture (Feature-First Approach) কাঠামোর ওপর ভিত্তি করে তৈরি হবে:



— entities/	# বিজনেস অবজেক্টস
— repositories/	# রিপোজিটরি ইন্টারফেস বা কন্ট্রাক্ট
— usecases/	# সুনির্দিষ্ট বিজনেস লজিক বা অ্যাকশন
— presentation/	# প্রজেন্টেশন লেয়ার
— bloc / provider/	# স্টেট ম্যানেজমেন্ট (Bloc/Cubit বা Riverpod)
— screens/	# ফুল স্ক্রিন ইউআই
— widgets/	# ফিচারে

৩. প্রজেক্টকে আধুনিক ও নিখুঁত করতে করণীয় পদক্ষেপসমূহ (Action Plan)

অ্যাপটিকে ইন্ডাস্ট্রি-স্ট্যান্ডার্ডে রূপান্তর করতে নিচের ৪টি ধাপে কাজ করতে হবে:

ধাপ ১: লেয়ার সেপারেশন ও মডুলার স্ট্রাকচার তৈরি করা

- বর্তমান ক্ল্যাট স্ট্রাকচার (**views/**, **controllers/**, **repositories/**) থেকে সরে এসে **Feature-First** বা **Clean Architecture** ফোল্ডার প্যাটার্নে রূপান্তর করতে হবে।
- ডোমেইন লেয়ারকে সম্পূর্ণ স্বাধীন রাখতে হবে, যাতে থার্ড-পার্টি প্যাকেজের পরিবর্তন হলেও বিজনেস লজিক অক্ষুণ্ণ থাকে।

ধাপ ২: ডিপেন্ডেন্সি ইনজেকশন (DI) ও স্টেট ম্যানেজমেন্ট ফিক্স করা

- সরাসরি ক্লাস ইনস্ট্যান্সিয়েশন বন্ধ করতে **get_it** এবং **injectable** প্যাকেজ যুক্ত করে একটি সেন্ট্রাল ডিপেন্ডেন্সি কন্ট্রোলার তৈরি করতে হবে।
- স্টেট ম্যানেজমেন্টের জন্য স্কেলেবল পদ্ধতি হিসেবে **Flutter Bloc / Cubit** অথবা **Riverpod** পুরোপুরি বাস্তবায়ন করতে হবে।

ধাপ ৩: ফাংশনাল এরর হ্যান্ডলিং ও পারফরম্যান্স অপটিমাইজেশন

- ক্র্যাশ বা আনহ্যান্ডেলড এক্সেপশন এড়াতে **fpdart** বা কাস্টম **Result<Success, Failure>** প্যাটার্ন ব্যবহার করে ফাংশনাল এরর হ্যান্ডলিং নিশ্চিত করতে হবে।
- মেমোরি লিক এড়াতে সমস্ত কন্ট্রোলার এবং স্ট্রিম সাবস্ক্রিপশন সঠিক নিয়মে **dispose()** করতে হবে এবং বড় লিস্টের ক্ষেত্রে **ListView.builder** ব্যবহার করতে হবে।

ধাপ ৪: টেস্টিং পাইপলাইন সেটআপ করা

- বিজনেস লজিক ও ইউজকেসগুলোর জন্য **bloc_test** এবং **mocktail** ব্যবহার করে ইউনিট টেস্ট (Unit Tests) লেখা শুরু করতে হবে, যা কোডের লং-টার্ম স্ট্যাবিলিটি নিশ্চিত করবে।